Post-Quantum Protection of QKD Key Delivery: the ETSI GS QKD 014 Interface

Sylvain Cormier
Paraxiom Technologies Inc.
sylvain@paraxiom.org

September 2025 (original), February 2026 (revised), November 2026 (corrected)

Abstract

Quantum Key Distribution (QKD) produces keys whose secrecy rests on physics. Those keys are then handed to the applications that use them through a key delivery interface, in most deployments the REST/HTTPS interface of ETSI GS QKD 014. That interface is protected by TLS, which today authenticates with RSA or elliptic curve signatures and, in most configurations, agrees its session keys with an elliptic curve exchange. Both are broken by Shor's algorithm.

This paper is about that delivery link. It is not about the classical channel that the two QKD units use between themselves for sifting, error correction and privacy amplification, which is authenticated separately and is outside our scope. We state precisely (Proposition 2) what follows for the delivered keys: an adversary who records delivery traffic today can recover the delivered keys once a quantum computer breaks the session's key exchange, and from that day can also impersonate a client or a key manager. The security of the quantum channel prevents neither.

We then describe the Post-Quantum Transport Gateway (PQTG), an open-source gateway that runs next to a key manager and replaces the exposed TLS endpoint with a post-quantum channel (ML-KEM-768 for key establishment, Falcon-512 signatures, AES-256-GCM), with no change to the vendor's equipment. We report what has been checked (symbolic models, tests, fuzzing, structural lemmas in Lean 4, one end-to-end run on a testbed) and what has not: there is no computational security proof of the protocol and no external audit, and the software is pre-stable.

Keywords: Quantum Key Distribution, post-quantum cryptography, ETSI GS QKD 014, key delivery, Shor's algorithm, ML-KEM, Falcon, harvest now, decrypt later

Corrections in this version (November 2026)

The earlier versions of this paper contained errors. This version replaces them.

- **Which channel.** Earlier versions said that deployed QKD systems authenticate the classical channel of the QKD protocol itself (basis reconciliation, error correction, privacy amplification) with TLS and RSA or ECDSA certificates. That merged two different links. The claim of this paper concerns key delivery only.
- **The main result.** It was stated as a theorem about the QKD protocol's authentication assumption. It is now a proposition about delivered keys, and it separates passive recording from active impersonation.
- **Security proofs.** The earlier "IND-CCA2" and "EUF-CMA" theorems for PQTG were not proofs and described certificates that the software does not use. They are removed. Section 6 says what has been checked.
- **Formal verification.** "649 machine-checked theorems with zero unresolved assumptions" did not describe this software. The gateway's own Lean development has 111 theorems about sizes and structure and one stated axiom.
- **Performance.** The table comparing with RSA-2048 was not measured on this software. It is replaced by measurements of the gateway's operations.
- **Other statements.** The list of named vendors as affected or supported, "the only deployed mitigation", the MIT licence, SPHINCS+-256f, the dependencies and the deployment commands were wrong or unsupported and have been corrected or removed.

Summary for non-specialist readers

The problem

A QKD system makes keys. Applications then fetch those keys from a key manager over an ordinary TLS connection, authenticated with classical certificates. A future quantum computer breaks that connection: it can decrypt recorded key deliveries and forge the certificates. **The keys are made with quantum security and delivered with classical security.**

Where it applies

Wherever keys leave a key manager over classical TLS and cross a network that an adversary can observe. ETSI GS QKD 014 specifies REST over HTTPS for key delivery, so the question arises for any deployment of that interface. How exposed a given deployment is depends on where the delivery link runs: inside one locked room it matters little, between sites it matters a great deal. We have not surveyed vendors or operators.

What PQTG is

A software gateway that runs on, or next to, the key manager. Clients reach it over a post-quantum channel; it reaches the vendor's interface over the local machine only. The vendor's equipment is not modified. It is open source (GPL-3.0, with a commercial licence available), pre-stable, and has not been externally audited.

Why timing matters

An adversary can record key delivery traffic **today**. Once a quantum computer breaks the key exchange of those TLS sessions, every key delivered in them is exposed, together with whatever those keys protected, if that was recorded too. This can only be closed before the traffic is recorded.

1 Introduction

1.1 Two classical links around a QKD system

Quantum Key Distribution [1,2] lets two parties establish a shared secret key with security that rests on quantum mechanics rather than on computational assumptions [3, 4].

Two classical links surround the quantum channel, and they must not be confused.

The protocol channel. The two QKD units exchange classical messages for sifting, parameter estimation, error correction and privacy amplification. QKD security proofs assume that this channel is authenticated [5]. The usual construction is information-theoretic authentication with a pre-shared symmetric key [6], refreshed from the keys the link produces. It does not rely on RSA or elliptic curves. How a given product implements it is the vendor's matter, and this paper makes no claim about it.

The delivery link. Once keys exist, applications obtain them from a key management entity through an application interface. The keys leave the QKD system on this link. It is the subject of this paper.

1.2 The ETSI GS QKD 014 delivery interface

ETSI GS QKD 014 [7] defines a REST interface over HTTPS through which a secure application entity retrieves keys from a key management entity. In the deployment we have worked with, this interface is protected by mutual TLS with X.509 client certificates.

TLS 1.2 and 1.3, as standardized and as deployed today, authenticate with RSA, ECDSA or EdDSA signatures. Post-quantum signatures in TLS certificates are not yet standardized for general use, and we know of no QKD key manager that offers them; we have not surveyed the market. The channel that *delivers* quantum keys is therefore protected by classical public-key cryptography. If a delivery session is broken, the keys it carried are known to the adversary, whatever the security of the quantum channel that produced them.

1.3 The gap

Shor's algorithm [8] factors integers and computes discrete logarithms in polynomial time on a quantum computer, which breaks RSA and elliptic curve cryptography. When such a computer will exist is uncertain [9]. Two distinct threats follow for the delivery link.

- **Record now, decrypt later.** An adversary records delivery sessions today. Later, a quantum computer recovers the session keys from the recorded key exchange and decrypts the responses that carried the QKD keys. This threat exists from the day of the recording.
- **Impersonation.** Once a quantum computer exists, an adversary can recover the private key behind a certificate and complete a TLS handshake as a client or as a key manager. This threat starts on that day and concerns sessions from then on.

Keys produced with information-theoretic security are delivered over a link whose security a quantum computer removes.

1.4 Contributions

1. A precise statement of what classical TLS on the delivery link means for the delivered keys (Proposition 2), kept separate from the QKD protocol's own classical channel.
2. The design of PQTG, a gateway that replaces the exposed TLS endpoint of a key manager with a post-quantum channel, without modifying the vendor's equipment.
3. An account of the evidence for the implementation, with its limits: symbolic models, tests, fuzzing, structural lemmas, measurements, and one end-to-end run with a commercial QKD system.

1.5 Related work

Post-quantum cryptography. NIST has standardized ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) [10]. FN-DSA, derived from Falcon, is to be FIPS 206 and is not final at the time of writing.

Hybrid TLS. Hybrid key exchange for TLS 1.3 [11], supported by the Open Quantum Safe project [12] among others, adds a post-quantum key exchange next to the classical one. Where both ends of a delivery link support it, it answers the record-now, decrypt-later threat. It leaves authentication classical.

QKD security in practice. Earlier work covers implementation weaknesses [13], attacks on devices [14], the authentication requirement of the protocol channel [5], and QKD networking [17]. We do not claim to be the first to observe that key delivery runs over classical TLS; our search of the literature was limited.

2 Preliminaries

2.1 QKD security

Definition 1 (QKD security)**.** *A QKD protocol is ϵ-secure if, for any adversary controlling the quantum channel, it either aborts or outputs keys K_A, K_B with $\Pr[K_A \neq K_B] \leq \epsilon_{cor}$ and with K_A ϵ_{sec}-close to uniform and independent of the adversary's quantum state.*

These proofs assume an authenticated protocol channel [4]. That assumption is not what this paper examines.

2.2 Quantum adversary

Definition 2 (Quantum adversary)**.** *A quantum adversary has polynomial-time access to a quantum computer.*

Theorem 1 (Shor [8])**.** *A quantum adversary can factor an RSA modulus, and can compute discrete logarithms in a finite field or on an elliptic curve, in time polynomial in the size of the instance.*

Resource estimates are of the order of a few thousand logical qubits; for example about 2330 for the discrete logarithm on a 256-bit elliptic curve [16]. The number of physical qubits needed is far larger, and no such machine exists today.

2.3 Post-quantum primitives used

A key encapsulation mechanism (KEM) is a triple $(\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$; a signature scheme is a triple $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$. PQTG uses ML-KEM-768 [10] (NIST security category 3) for key establishment and Falcon-512 [15] (category 1) for signatures. Authentication is therefore at category 1, the lower of the two. An SLH-DSA-SHAKE-128f key (category 1) signs the gateway's audit log checkpoints and signs nothing in the handshake.

3 System model and threats

3.1 System

A QKD link consists of two QKD units, a quantum channel and the protocol channel between them. Each site has a key management entity that stores the keys and serves them through the delivery interface. An application

calls, for example, `/api/v1/keys/{slave_SAE_ID}/enc_keys` over HTTPS, authenticated by mutual TLS, and receives key material with a key identifier; its peer retrieves the same key by that identifier from the key manager at the other site.

3.2 Adversary

We consider $\mathcal{A} = (\mathcal{A}_{\text{now}}, \mathcal{A}_{\text{future}})$.

- $\mathcal{A}_{\text{now}}$ observes and records traffic on the delivery link. It cannot break RSA or elliptic curve cryptography.
- $\mathcal{A}_{\text{future}}$, from a time t_1, has a quantum computer, the recordings, and network access to the delivery link.

The adversary does not control the key manager or the application host. An adversary who does needs no cryptanalysis.

4 What classical TLS means for delivered keys

Proposition 2 (Delivered keys inherit the security of the delivery session)**.** *Let a key K be produced by a QKD link and delivered to an application in a TLS session whose key exchange and authentication use RSA or elliptic curve cryptography.*

1. *(Passive.) If $\mathcal{A}_{now}$ records the session, then $\mathcal{A}_{future}$ recovers K.*
2. *(Active.) From t_1 on, $\mathcal{A}_{future}$ can impersonate a client to the key manager, or the key manager to a client, and so obtain keys or substitute them.*

Both hold whatever the security of the QKD protocol that produced K.

Proof. (1) The session's traffic keys are derived from an RSA key transport or from a Diffie-Hellman exchange, over a finite field or an elliptic curve, whose public values are in the recorded handshake. By Theorem 1, $\mathcal{A}_{\text{future}}$ recovers the shared secret, derives the traffic keys and decrypts the response that carries K.

(2) By Theorem 1, $\mathcal{A}_{\text{future}}$ recovers the private key of a certificate, or of the authority that issued it, from the public key, and completes a TLS handshake as that party. □

Corollary 3. *A QKD deployment that delivers keys over TLS with RSA or elliptic curve cryptography, on a link the adversary can observe, does not provide quantum-safe keys to its applications, even when its quantum channel is secure.*

Remark 1. *The statement follows directly from Shor's algorithm; its use is to say exactly what is and is not affected. It does not depend on the QKD protocol or the vendor. It says nothing about the protocol channel between the QKD units. Part 1 does not apply to a delivery session that uses a post-quantum or hybrid key exchange; part 2 applies as long as authentication is classical.*

5 The Post-Quantum Transport Gateway

5.1 Design

PQTG runs on the key manager's machine, or on a machine next to it on a link the adversary cannot observe. It has two sides.

- **Outside.** It listens for clients on a post-quantum channel of its own. This is not TLS.
- **Inside.** It calls the vendor's ETSI GS QKD 014 interface over classical TLS on the loopback interface, as a normal client of that interface.

The vendor's equipment is not modified. The protection holds only if the vendor's interface is in fact unreachable from the network, which is a matter of the host's configuration and outside what the gateway can enforce.

There are no certificates and no certificate authority on the post-quantum channel. The gateway admits the clients whose public keys are listed in a local file, and a client is expected to compare the gateway's public key with a fingerprint it obtained beforehand, as with SSH.

5.2 Handshake

The current wire version is 3.2. In outline:

1. The client generates a fresh ML-KEM-768 key pair for the session and sends a hello with the encapsulation key, its Falcon-512 public key, a time stamp and, if it wants a QKD key mixed in, its application identifier. The client signs the hello with its Falcon-512 key.

2. The gateway checks that the client's key is on its list, verifies the signature, and refuses a hello that is too old or already seen.

3. If the client asked for it and the key manager can serve it, the gateway obtains a QKD key and its identifier from the key manager.

4. The gateway encapsulates to the client's ML-KEM key, and signs with its Falcon-512 key a transcript that covers both hellos, the ciphertext, the key mode and the QKD key identifier.

5. The client verifies the signature against the gateway key it expects, decapsulates, and in hybrid mode retrieves the same QKD key by its identifier from the key manager on its own side.

6. Both ends derive the session key from the ML-KEM shared secret, the QKD key when present, and the transcript, using a domain-separated schedule based on HMAC-SHA3-256.

The session key is expanded into one AES-256-GCM key per direction. Each direction counts its own nonces, and each end accepts only the next expected frame. Inside a session, each request and response is also signed with Falcon-512. The exact fields and the key schedule are specified in the repository (`docs/PROTOCOL-V3-QKD-KEYID.md`, `docs/KEY-SCHEDULE.md`).

Hybrid mode requires a client that can reach the key manager on its own side. In the repository today, the gateway side of hybrid mode is implemented; a complete reference client for it is not.

5.3 Other functions

The gateway keeps an audit log of connections, key requests and refused clients. The log is hash-chained and the gateway signs checkpoints with its SLH-DSA key. The signing key is a file on the same host, so the log detects changes made without that key, not changes made by whoever controls the host.

6 What has been checked, and what has not

There is no computational (reduction-based) security proof of the PQTG protocol. The security argument is the following, with the evidence for each part.

Assumptions. ML-KEM-768 is IND-CCA2 secure; Falcon-512 is unforgeable; HMAC-SHA3-256 behaves as a pseudorandom function; AES-256-GCM is a secure authenticated cipher; and the implementations used (the `paraxiom-pqc` crate and the RustCrypto crates) are correct. These are assumed, not derived, and `paraxiom-pqc` has not been externally audited.

Symbolic analysis. The handshake, the client-signed hello, the session layer and the hybrid mode are modelled in Verifpal against an active attacker. In those models the session key stays confidential and the server is authenticated to the client, and the session key stays confidential when either the ML-KEM secret or the QKD key is given to the attacker, but not both. The model's query on authentication of the client's hello reports a replay: the model has no clocks and no server state, so it cannot express the time stamp and the replay guard with which the code refuses a hello it has already seen. A replayed hello exposes no secret. Verifpal is a symbolic, bounded tool: a pass means no attack was found within the session bound, under an idealized view of the primitives. The models, their results and their limits are in the repository under `formal/`.

Tests. Unit, integration and property tests cover the handshake, the session layer and the ETSI GS QKD 014 client; the latter runs against an emulator of the interface. Fuzzing of four targets for 240 seconds each (about 5.2 million executions) found no crash. Mutation testing of the session code found gaps that were then closed with new tests. Of seven bounded model checking harnesses (Kani), one reached a verdict.

Lean 4. The repository contains 111 machine-checked theorems. They concern sizes and structure: byte lengths, framing limits, the shape of the transcript, parameter values against the FIPS documents. They do not establish any security property, and the development declares one axiom, on agreement of the KEM's two shared secrets in its abstract model.

Forward secrecy. The ML-KEM key pair is fresh for each session, so a later compromise of the signing keys does not expose past session keys under the assumptions above. This is argued, not proved.

Not done. No external audit. No side-channel evaluation beyond timing tests of the underlying library. No validation under any certification scheme: the cryptographic implementations are not validated, Falcon-512 is not yet a final standard, and nothing in this paper should be read as a claim of compliance.

6.1 Known limits

The repository's threat model (`docs/THREAT-MODEL.md`) lists the limits known to us. The main ones:

- A client must check the gateway's key fingerprint itself. A client that does not is exposed to an attacker in the middle.
- Removing a client means editing the list of keys and restarting the gateway; there is no revocation mechanism.
- Protection against denial of service is limited to connection limits and timeouts.
- The signature inside a session covers the message only, not its session, direction or position. If one direction key leaks, a request the client really signed can be replayed.
- The software is pre-stable. An earlier release reused AES-GCM nonces across the two directions of a session; it was withdrawn, and the advisory is in `SECURITY.md`.

7 Implementation and measurements

7.1 Implementation

PQTG is written in Rust. The post-quantum primitives come from the `paraxiom-pqc` crate; AES-GCM, SHA-3 and HMAC from the RustCrypto crates; the asynchronous runtime is `tokio`. The connection to the vendor's interface uses a TLS library (`rustls`) whose cryptographic backend contains C and assembly code; that code is used only on the loopback side.

Source: https://github.com/Paraxiom/pq-transport-gateway. Licence: GPL-3.0-only, or a commercial licence from Paraxiom.

7.2 Measurements

Table 1 gives the cost of the gateway's cryptographic operations, measured on 8 September 2026 with `criterion` on an Apple Silicon development machine. They are single-machine microbenchmarks, not throughput figures for a deployed gateway.

With the identity key generated once, the cryptographic cost of accepting a connection is well under one millisecond on this machine. The signature

Operation	Primitive	Time
Identity key generation (once)	Falcon-512, SLH-DSA-SHAKE-128f	7.27 ms
Session key pair generation	ML-KEM-768	33.0 μs
Encapsulation	ML-KEM-768	31.7 μs
Decapsulation	ML-KEM-768	66.2 μs
Transcript signature	Falcon-512	326 μs
Transcript verification	Falcon-512	30.0 μs
Transcript hash (about 3.2 KB)	SHA3-256	5.5 μs

Table 1: Cost of the gateway's cryptographic operations (Apple Silicon, one core).

on every message inside a session costs about 0.36 ms per message for signing and verification together, which bounds the message rate of one core; removing it is planned (Section 9.1). We make no comparison with classical TLS: we have not measured one under the same conditions.

7.3 Testbed run

The gateway's ETSI GS QKD 014 client passes its nine-item conformance suite against our reference emulator. On a Canadian quantum communication testbed, in June 2026, a key was retrieved through the delivery interface of a commercial QKD system, and the key with the same identifier, retrieved at the peer key manager, was identical. This is one vendor and one recorded run. Equipment from other vendors on the same testbed was reachable, but we completed no key delivery with it.

8 Discussion

8.1 Hybrid TLS

Hybrid TLS adds a post-quantum key exchange to TLS 1.3 and keeps classical certificates. Where a key manager and its clients support it, it removes the record-now, decrypt-later threat (part 1 of Proposition 2), which is the more urgent of the two. It does not remove impersonation by an adversary who has a quantum computer (part 2); that needs post-quantum authentication on both sides of a mutually authenticated link. PQTG addresses both parts, at the price of a non-standard protocol on the outside.

8.2 Alternatives

PQTG is one way to protect the delivery link. Others exist or will exist:

- **Vendor support.** A key manager that offers post-quantum key exchange and, later, post-quantum certificates on its own interface needs no gateway. We do not know vendors' plans.
- **A tunnel.** The delivery link can be carried in a VPN that uses post-quantum key exchange or pre-shared symmetric keys. This protects the link without a purpose-built gateway; it does not by itself give per-application authentication at the key manager.
- **Pre-shared symmetric keys.** TLS with pre-shared keys avoids public-key cryptography altogether, at the cost of distributing and rotating those keys.
- **Physical separation.** Where the application and the key manager share a protected enclosure, the delivery link is not observable and the question does not arise.

8.3 Limitations of the approach

PQTG assumes that the post-quantum algorithms it uses are secure; advances in cryptanalysis could require changing them, and the current version fixes its algorithms at build time. Its outside protocol is its own and has had far less scrutiny than TLS. The gateway is an additional component to operate, and a host that runs it becomes part of the trusted base of the key manager.

9 Conclusion

QKD keys are delivered to applications over classical TLS. A quantum computer removes the security of that link, first for recorded sessions and then for live ones, whatever the security of the quantum channel. This concerns key delivery; it does not concern the protocol channel between the QKD units.

PQTG is an open-source gateway that replaces the exposed delivery endpoint with a post-quantum channel without modifying the vendor's equipment. It has been modelled symbolically, tested, fuzzed, and run once end to end with a commercial QKD system. It has no computational security

proof and no external audit, and it is pre-stable. It is one of several ways to close the gap.

9.1 Future work

- A profile with ML-DSA-65 for authentication, next to Falcon-512, with the choice bound into the signed handshake.
- Signing only in the handshake, with authenticated encryption alone for messages inside a session.
- Keeping the gateway's keys in a hardware security module.
- A revocation mechanism for client and gateway keys.
- A complete reference client, including the client side of hybrid mode.
- Algorithm validation of the cryptographic library as a first step towards any certification, and an external audit.

Acknowledgments

We thank the NIST Post-Quantum Cryptography project and the Open Quantum Safe project for foundational work. We thank the operators of the Canadian quantum communication testbed on which the run of Section 7.3 was made, for access to real QKD equipment.

This corrected version was prepared with the help of an AI assistant. The author checked each statement against the source code, the test records and the measurements cited, and is responsible for the content.

References

[1] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," *Proc. IEEE Int. Conf. Computers, Systems and Signal Processing*, pp. 175–179, 1984.

[2] A. K. Ekert, "Quantum cryptography based on Bell's theorem," *Phys. Rev. Lett.*, vol. 67, no. 6, p. 661, 1991.

[3] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, "Quantum cryptography," *Rev. Mod. Phys.*, vol. 74, no. 1, p. 145, 2002.

[4] P. W. Shor and J. Preskill, "Simple proof of security of the BB84 quantum key distribution protocol," *Phys. Rev. Lett.*, vol. 85, no. 2, p. 441, 2000.

[5] V. Scarani et al., "The security of practical quantum key distribution," *Rev. Mod. Phys.*, vol. 81, no. 3, p. 1301, 2009.

[6] M. N. Wegman and J. L. Carter, "New hash functions and their use in authentication and set equality," *J. Comput. Syst. Sci.*, vol. 22, no. 3, pp. 265–279, 1981.

[7] ETSI, "Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API," *ETSI GS QKD 014 V1.1.1*, 2019.

[8] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[9] M. Mosca, "Cybersecurity in an era with quantum computers: will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[10] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard; Module-Lattice-Based Digital Signature Standard; Stateless Hash-Based Digital Signature Standard," *FIPS 203, 204, 205*, 2024.

[11] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid key exchange in TLS 1.3," *IETF Internet-Draft*, draft-ietf-tls-hybrid-design.

[12] D. Stebila and M. Mosca, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," *Int. Conf. Selected Areas in Cryptography*, pp. 1–24, 2016.

[13] F. Xu, X. Ma, Q. Zhang, H. K. Lo, and J. W. Pan, "Secure quantum key distribution with realistic devices," *Rev. Mod. Phys.*, vol. 92, no. 2, p. 025002, 2020.

[14] V. Makarov et al., "Creation of backdoors in quantum communications via laser damage," *Phys. Rev. A*, vol. 94, no. 3, p. 030302, 2016.

[15] T. Pornin, "New efficient, constant-time implementations of Falcon," *IACR Cryptology ePrint Archive*, Report 2019/893, 2019.

[16] M. Roetteler, M. Naehrig, K. M. Svore, and K. Lauter, "Quantum resource estimates for computing elliptic curve discrete logarithms," *Advances in Cryptology – ASIACRYPT 2017*, pp. 241–270, 2017.

[17] M. Mehic et al., "Quantum key distribution: a networking perspective," *ACM Comput. Surv.*, vol. 53, no. 5, pp. 1–41, 2020.

A Running the gateway

Build and generate the gateway's identity:

```
git clone https://github.com/Paraxiom/pq-transport-gateway
cd pq-transport-gateway
cargo build --release
sudo ./target/release/pq-qkd-proxy --generate-keys
```

Configuration (`/etc/pq-qkd-proxy/config.toml`):

```
[proxy]
listen = "192.168.1.100:8443"

[qkd]
vendor_api  = "https://localhost:443"
vendor_cert = "/etc/qkd/vendor-cert.pem"

[security]
pq_algorithm    = "falcon512"
sig_algorithm   = "slh-dsa-shake128f"
authorized_keys = "/etc/pq-qkd-proxy/authorized_keys"
```

The vendor's interface must not be reachable from the network; how to ensure that depends on the host and on the vendor's equipment. The repository's README and `docs/CLIENT-INTEGRATION.md` are the reference for current options.

B Security categories

NIST defines its post-quantum security categories by comparison with symmetric primitives: breaking a category 1 scheme should be at least as hard as key search on AES-128, category 3 as key search on AES-192, and category 5 as key search on AES-256. PQTG uses ML-KEM-768 (category 3) for key establishment and Falcon-512 (category 1) for authentication.